SMARTCAMPUS — AI CONTEXT HANDOFF

Résidences Module — Full Project Context

Everything an AI assistant needs to understand, extend or debug the housing module: project, architecture, contracts, conventions, state and roadmap.

NestJS 10 · TypeORM · PostgreSQL | Vite + React 19 · Tailwind | Chambre · Demande · Contrat | Mock + real modes

> **How to use this document.** Treat the **code as the source of truth**; this document reflects the repository as of **2026-11**. Entity, status and role names appear in their original **French** form because they are used verbatim in JSON payloads and database enums. A Markdown twin (`SmartCampus-Residences-AI-Context.md`) sits next to this PDF.

1. Project overview

SmartCampus is a campus-management platform built as a microservices student project (ESPRIT 5SE3). It is split into six functional modules:

Module	Owner	Frontend folder	Backend
Utilisateurs (auth, profiles, roles)	teammate	`src/features/users/`	external `IAuthentification` service
Académique	teammate	`src/features/academic/`	not in this repo
Ressources (room reservations)	teammate	`src/features/resources/`	not in this repo
Résidences (this work)	**this work**	`src/features/housing/`	`backend/` **NestJS (this work)**
Vie étudiante	teammate	`src/features/student-life/`	not in this repo
Bibliothèque	teammate	`src/features/library/`	not in this repo

- **Frontend reality:** the shared app is **Vite + React 19 + React Router 7 + Tailwind** — *not* Next.js. It is mock-driven by default (`VITE_USE_MOCKS`); teammates' modules only have mock data.
- **Auth:** the platform consumes `IAuthentification` (JWT) from the Utilisateurs service. Currently the frontend issues a **fake JWT** (`src/auth/jwt.ts`) stored in `localStorage["smartcampus.jwt"]` ; the shared API client injects it as `Authorization: Bearer …` . Demo users per role (`src/auth/roles.ts`): student `ETU-2024-0187` , teacher `ENS-0042` , admin `ADM-0001` .
- **Roles:** `student | teacher | admin` . Résidences is available to **student** and **admin** only (teacher → 403 via the module registry).

2. Repository map

```
D:\Microservices\SMART-COMPUS-ESPRIT-5SE3-FRONT-END\
├── backend/                    ← NEW (this work): NestJS API, 3 services
├── docs/                       ← NEW (this work): documentation PDFs/Markdown/HTML
├── src/                        ← team's shared React app (Vite)
│   ├── api/client.ts           shared fetch client: base URL, JWT interceptor, mock mode
│   ├── api/useQuery.ts         minimal data-fetching hook (loading/error/refetch)
│   ├── api/useMutation.ts      minimal mutation hook
│   ├── auth/                   AuthContext, roles, fake JWT, RequireSession
│   ├── components/ui/          design-system primitives (button, card, dialog, select…)
│   ├── components/shared/      DataTable, StatCard, StatusPill, QueryBoundary, EmptyState…
│   ├── features/registry.ts    module registry (id, path, icon, roles, lazy Page)
│   ├── features/housing/       ← THIS MODULE (rewritten to the backend contract)
│   ├── i18n/                   FR dictionary + registerMessages("fr", {…}) per feature
│   └── features/dashboard/, home/, users/, academic/, resources/, student-life/, library/
├── .env.example                VITE_API_BASE_URL (default gateway :8080), VITE_USE_MOCKS
├── package.json                scripts: dev | build (tsc -b && vite build) | typecheck
└── README.md / architecture.md
```

Scope discipline: only `src/features/housing/**` (plus `docs/`) was touched in the frontend. The registry, navigation and routes in `App.tsx` are unchanged — the registry already routed `/residences/*` to the housing module.

3. System architecture

```
[ React SPA (Vite) ]
      │  fetch JSON + Bearer token (shared interceptor in src/api/client.ts)
      ▼
[ NestJS API : http://localhost:3001/api ]   (Swagger UI at /api/docs)
      │
      ├── ChambresController ─► ChambresService ─► TypeORM ─┐
      ├── DemandesController ─► DemandesService ─► TypeORM ─┼─► PostgreSQL
      └── ContratsController ─► ContratsService ─► TypeORM ─┘    schema "BDD_Residences"
                                      │
                        EventEmitter2 (in-process bus)
                          ├ "demande.approuvee" ─► ContratsService.createFromDemande()
                          └ "demande.annulee"   ─► ContratsService résilie le contrat
                        Cross-service calls by NestJS DI:
                          • DemandesService ─► ChambresService.occuper()/liberer()
                          • ContratsService ─► ChambresService.findOne()/liberer()
```

- **Design decision (approved):** the three services share one PostgreSQL database and are wired with **NestJS dependency injection** (not HTTP). The event bus decouples Demande → Contrat. Rules stay testable and the services can be extracted as real HTTP microservices later without moving business logic.
- **API style:** one Nest app, global prefix `api`, global validation pipe, Swagger documentation.

4. Backend reference (authoritative contract)

4.1 Stack and libraries (`backend/package.json`)

Library	Role
`@nestjs/{core,common,platform-express}` 10	HTTP framework, DI, controllers
`@nestjs/typeorm` + `typeorm` + `pg`	ORM + PostgreSQL driver
`@nestjs/config`	typed `.env` loading
`@nestjs/swagger`	OpenAPI docs at `/api/docs`
`@nestjs/event-emitter` 2	in-process domain events
`class-validator` + `class-transformer`	DTO validation / transformation
Jest + `@nestjs/testing`	unit tests (36 total)

Scripts: `npm run start:dev` · `npm run build` · `npm test` · `npm run lint` . Port **3001**, prefix `/api` , schema `BDD_Residences` .

4.2 Cross-cutting files

File	Description
`src/main.ts`	`ensureSchemaExists()` first; global `ValidationPipe({ whitelist, forbidNonWhitelisted, transform, enableImplicitConversion })` ; CORS; Swagger at `/api/docs` with Bearer auth definition
`src/app.module.ts`	`ConfigModule.forRoot({ isGlobal })` ; `TypeOrmModule.forRootAsync` (autoLoadEntities, synchronize/logging from env); `EventEmitterModule.forRoot()` ; imports Chambres/Demandes/Contrats modules
`src/config/database.config.ts`	`registerAs("database")` : host, port, username, password, database, schema, synchronize, logging
`src/config/ensure-schema.ts`	raw `pg` client runs `CREATE SCHEMA IF NOT EXISTS "<DB_SCHEMA>"` before TypeORM connects (TypeORM does not create schemas)
`src/common/transformers/numeric.transformer.ts`	PostgreSQL `numeric` (returned as string) → `number` in API responses
`src/common/decorators/public.decorator.ts`	`@Public()` metadata for the future global auth guard
`src/common/guards/jwt-auth.guard.ts`	**Placeholder** guard: allows requests, only checks Bearer format; not registered globally. TODO: verify JWT from `IAuthentification`

Environment variables (`backend/.env` , see `.env.example`): `NODE_ENV` , `PORT=3001` , `API_PREFIX=api` , `DB_HOST` , `DB_PORT` , `DB_USER` , `DB_PASSWORD` , `DB_NAME=smartcampus` , `DB_SCHEMA=BDD_Residences` , `DB_SYNCHRONIZE=true` , `DB_LOGGING=false` , `JWT_SECRET` , `JWT_ISSUER` .

`docker-compose.yml` : Postgres 16-alpine (db `smartcampus` , user/password `postgres`), healthcheck, volume, mounts `scripts/init-schema.sql` into `docker-entrypoint-initdb.d` . **Docker is not installed on the current dev machine** — a local PostgreSQL or Docker is required for a real end-to-end run.

4.3 Chambre Service (Phase 1 — foundation)

Entity `Chambre` (table `chambres` , unique `UQ_chambre_batiment_numero` on `(batiment, numero)`):

Field	Type	Constraints
`id`	uuid PK	generated
`numero`	varchar(50)	not empty
`type`	enum `ChambreType`	`SIMPLE | DOUBLE | STUDIO`
`capacite`	int	1–4; must match type: SIMPLE=1, DOUBLE=2, STUDIO 1–2 (400 otherwise)
`disponible`	boolean	default `true`
`batiment`	varchar(100)	unique with `numero`
`etage`	int	≥ 0
`createdAt` / `updatedAt`	timestamptz	auto

Endpoints (UUID path params via `ParseUUIDPipe`):

Method	Route	Purpose	Errors
POST	`/api/chambres`	create	400 incoherent capacity, 409 duplicate
GET	`/api/chambres?type=&batiment=&disponible=`	list with filters	—
GET	`/api/chambres/disponibles?type=&batiment=`	only free rooms	—
GET	`/api/chambres/:id`	detail	404
PATCH	`/api/chambres/:id`	partial update	400 / 404 / 409
DELETE	`/api/chambres/:id`	delete	404

Service extras (used by the other services):

- `occuper(id)` — **atomic occupation**: `repository.update({ id, disponible: true }, { disponible: false })` ; if `affected === 0` , a `findOne` distinguishes 404 (missing) from **409 "déjà occupée"**. Prevents double assignment under concurrency.
- `liberer(id)` — idempotent release (`disponible = true`).
- `findAll` orders by `batiment ASC, numero ASC` ; the filter DTO transforms `"true"/"false"` query strings into booleans.

4.4 Demande Service (Phase 2)

Entity `DemandeHebergement` (table `demandes_hebergement`):

Field	Type	Notes
`id`	uuid PK	
`etudiantId`	varchar(100), indexed	from JWT later; passed in body today
`statut`	enum `DemandeStatut`, indexed	`EN_ATTENTE | APPROUVEE | REFUSEE | ANNULEE`
`dateDepot`	timestamptz	default now
`dateArrivee`, `dateDepart`	timestamptz	`dateArrivee < dateDepart` required (400)
`chambreId`	uuid nullable, indexed	wished / assigned room
`createdAt` / `updatedAt`	timestamptz	auto

State machine (any other transition, including to the same status → **409**):

```
EN_ATTENTE ──► APPROUVEE   (requires a free room; occupies it; creates the contract via event)
EN_ATTENTE ──► REFUSEE
EN_ATTENTE ──► ANNULEE
APPROUVEE  ──► ANNULEE     (frees the room; resiles the contract via event)
REFUSEE, ANNULEE: terminal
```

Endpoints:

Method	Route	Purpose	Errors
POST	`/api/demandes`	body `{ etudiantId, dateArrivee, dateDepart, chambreId? }`	400 dates, 404 unknown chambreId
GET	`/api/demandes?statut=&etudiantId=&chambreId=`	ordered `dateDepot DESC`	—
GET	`/api/demandes/:id`	detail	404
PATCH	`/api/demandes/:id/statut`	body `{ statut, chambreId? }`	400 (chambreId on non-approval), 404, 409

Approval flow: resolve room (`dto.chambreId ?? demande.chambreId`, else 400) → `ChambresService.occuper()` → save demande `APPROUVEE` → emit `demande.approuvee` with `{ demandeId, etudiantId, chambreId, dateDebut=dateArrivee, dateFin=dateDepart }`.

Cancellation flow (`APPROUVEE → ANNULEE`): set statut → `ChambresService.liberer()` → emit `demande.annulee` `{ demandeId, chambreId }`.

Events are defined in `src/demandes/events/demande.events.ts` (constants + payload classes) and emitted with `EventEmitter2.emit()`; listeners are async and self-catching.

4.5 Contrat Service (Phase 3)

Entity `Contrat` (table `contrats`, **unique** `UQ_contrat_demande` on `demandeId`):

Field	Type	Notes

`id`	uuid PK	
`residentId`	varchar(100), indexed	= demande.etudiantId
`chambreId`	uuid, indexed	
`demandeId`	uuid, indexed, unique	one contract per demande
`dateDebut` , `dateFin`	timestamptz	`dateFin > dateDebut` ; `dateFin ≤ academic year end`
`montantMensuel`	numeric(10,2)	transformer returns number; TND
`statut`	enum `ContratStatut` , indexed	`ACTIF | TERMINE | RESILIE`

Business rules:

- **Idempotent creation**: if a contract already exists for the demande, log a warning and return it (no duplicate even if the event fires twice).
- **Pricing** (`src/contrats/constants/pricing.constants.ts`): `SIMPLE 150` , `DOUBLE 100` , `STUDIO 200` TND/month — keyed by the room type fetched via `ChambresService.findOne()` .
- **Academic year end** (`src/contrats/utils/academic-year.util.ts`): July 31 (23:59:59.999 UTC) of the academic year starting Aug 1. Start in Aug–Dec → next year's July 31; Jan–Jul → same year's July 31. Violation → 400.
- **Resiliation**: only `ACTIF` (409 otherwise); sets `RESILIE` , then `ChambresService.liberer(chambreId)` .
- **Event listeners** (`@OnEvent(DEMANDE_APPROUVEE, { async: true, suppressErrors: true })` , same for `DEMANDE_ANNULEE`): wrapped in `try/catch` with `Logger.error` , so a contract failure **never breaks the demande approval response** (eventual consistency — a retry/outbox is a roadmap item).

Endpoints:

Method	Route	Purpose	Errors
GET	`/api/contrats?statut=&residentId=&chambreId=&demandeId=`	admin list (ordered `dateDebut DESC`)	—
GET	`/api/contrats/resident/:residentId?statut=`	a resident's contracts	—
GET	`/api/contrats/:id`	detail	404
PATCH	`/api/contrats/:id/resilier`	resile + free room	404, 409

4.6 Backend file tree (complete)

```
backend/
├── docker-compose.yml              Postgres 16 + init-schema mount
├── scripts/init-schema.sql         CREATE SCHEMA IF NOT EXISTS "BDD_Residences"
├── .env / .env.example             all env vars
├── package.json, tsconfig.json, tsconfig.build.json, nest-cli.json,
│   .eslintrc.js, .prettierrc, .gitignore
└── src/
    ├── main.ts                     bootstrap (schema, pipe, CORS, Swagger, port)
    ├── app.module.ts               config + TypeORM + events + 3 modules
    ├── config/database.config.ts   typed DB config
    ├── config/ensure-schema.ts     schema bootstrap
    ├── common/
    │   ├── decorators/public.decorator.ts
    │   ├── guards/jwt-auth.guard.ts          (placeholder)
    │   └── transformers/numeric.transformer.ts
    ├── chambres/
```

```
│   ├── chambres.module.ts / chambres.controller.ts / chambres.service.ts
│   ├── chambres.service.spec.ts          (11 tests)
│   ├── entities/chambre.entity.ts, entities/chambre-type.enum.ts
│   └── dto/create-chambre.dto.ts, update-chambre.dto.ts, filter-chambre.dto.ts
├── demandes/
│   ├── demandes.module.ts / demandes.controller.ts / demandes.service.ts
│   ├── demandes.service.spec.ts          (12 tests)
│   ├── entities/demande-hebergement.entity.ts, entities/demande-statut.enum.ts
│   ├── dto/create-demande.dto.ts, update-demande-statut.dto.ts, filter-demande.dto.ts
│   └── events/demande.events.ts          (names + payload classes)
└── contrats/
    ├── contrats.module.ts / contrats.controller.ts / contrats.service.ts
    ├── contrats.service.spec.ts          (13 tests)
    ├── entities/contrat.entity.ts, entities/contrat-statut.enum.ts
    ├── constants/pricing.constants.ts
    ├── utils/academic-year.util.ts
    └── dto/filter-contrat.dto.ts, find-resident-contrats.dto.ts
```

5. Frontend reference

5.1 Stack

React 19, React Router 7, Vite 8, TypeScript strict (`noUnusedLocals` , `noUnusedParameters`), Tailwind 3 + `tailwindcss-animate` , zod 4 (forms), sonner (toasts), recharts, Radix primitives, lucide-react, class-variance-authority. Scripts: `npm run dev` (port 5173), `npm run build` (`tsc -b && vite build`), `npm run typecheck` . **No frontend test harness.**

5.2 Routes and pages (module `housing` , registry path `/residences` , student + admin)

Route	View	Notes
`/residences`	Dashboard	Stats (rooms/free/occupied/pending demandes or my demandes), occupancy per bâtiment, latest demandes, active contracts; student CTA to create a demande
`/residences/chambres`	Room list + filters (type, bâtiment, disponible)	Admin: create (dialog), toggle `disponible` , delete (confirm)
`/residences/demandes`	**student**: my demandes (status, stay dates, assigned room, cancel if EN_ATTENTE/APPROUVEE)	**admin**: full queue, statut filter, counters, approve (pick a free room) / refuse
`/residences/demandes/new`	Demande form	Arrival/departure dates (validated `arrivee < depart`), optional preferred room; student only (403 otherwise)
`/residences/contrats`	**student**: my contracts (room, period, rent, status)	**admin**: all contracts, status filter, resile (frees room)
`/residences/maintenance`	Tickets (Signalé → En cours → Résolu)	**Mock-only service**, outside the 3 backends; real mode throws 501 with an explicit message

Tabs are role-aware in `pages/HousingModule.tsx` (nested `<Routes>` ; "Demandes" label differs student vs admin).

5.3 API layer (`src/features/housing/api/index.ts`)

Each function calls the real NestJS route via the shared client **and** provides a `mock` used when `VITE_USE_MOCKS !== "false"` :

Function	Backend route
`listChambres(filter, viewer, signal)`	GET `/chambres?type=&batiment=&disponible=`
`listChambresDisponibles(filter, …)`	GET `/chambres/disponibles`
`createChambre` / `updateChambre` / `deleteChambre`	POST / PATCH / DELETE `/chambres[/:id]`
`listDemandes(filter, viewer, signal)`	GET `/demandes` (non-admin forced to `etudiantId = viewer.id`)
`getDemande(id)`	GET `/demandes/:id`

`createDemande(input, viewer)`	POST `/demandes`
`updateDemandeStatut(id, statut, chambreId?, viewer)`	PATCH `/demandes/:id/statut`
`cancelDemande` / `refuseDemande` / `approveDemande`	wrappers over `updateDemandeStatut`
`listContrats` / `listContratsResident` / `getContrat`	GET `/contrats`, `/contrats/resident/:id`, `/contrats/:id`
`resilierContrat(id, viewer)`	PATCH `/contrats/:id/resilier`
`listTickets` / `createTicket` / `updateTicketStatus`	mock only; real mode throws `ApiError(501)`

The mock layer mirrors backend rules: atomic occupation (in-memory), transition table, contract creation with pricing + academic-year guard, release + resile on cancellation, and `publishDomainEvent()` toasts (`DemandeDéposée`, `DemandeApprouvée`, `ContratCréé`, `DemandeRefusée/Annulée`, `ContratRésilié`). Seed data: 28 rooms across `Bâtiment A/B/C`, 6 demandes (`d-1..d-6`), 2 active contracts (`k-1`, `k-2` for rooms `A-002` and `B-001`), 5 tickets. Demo student `ETU-2024-0187` (Yasmine) has a pending demande to exercise the flow.

5.4 Modes, auth, conventions

- **Mock mode** (default): `VITE_USE_MOCKS` unset. **Real mode**: `VITE_USE_MOCKS=false` and `VITE_API_BASE_URL=http://localhost:3001/api` (shared default is the team gateway `http://localhost:8080/api`).
- **Client** (`src/api/client.ts`): base URL + request interceptor (Bearer token from localStorage) + error normalisation into `ApiError(status, message, details)`; `onError` clears the token on 401.
- **Data fetching**: `useQuery(fetcher, deps)` → `{ data, error, isLoading, refetch }`, rendered through `<QueryBoundary>` (skeleton / service-unavailable / generic error + retry). Mutations via `useMutation` (throws `ApiError` for local toasts).
- **i18n**: French UI; keys registered via `registerMessages("fr", { "housing.…": … })`; all strings through `useT()`.
- **Design system**: `Card`, `Button`, `Dialog`, `Select`, `Input`, `DataTable` (sort/paginate/empty), `StatCard`, `EmptyState`, `ConfirmDialog`, `ConflictAlert`, navy/violet theme. Housing-local badges: `DemandeStatusBadge`, `ContratStatusBadge`, `TicketStatusBadge`.
- **Types**: `types/index.ts` mirrors backend JSON exactly (French enum values). `CHAMBRE_TYPES`, `MONTANT_MENSUEL_PAR_TYPE`, `capaciteValide()` centralise shared constants.
- **Schemas**: zod in `schemas/index.ts` (demande dates, room form incl. capacity-coherence refine, ticket). Messages are i18n keys returned via `fieldErrors()`.

5.5 Frontend file tree (module)

```
src/features/housing/
├── index.ts                  ModuleDef (unchanged: /residences, student+admin)
├── README.md                 (updated: routes, API contract, rules, backend hookup)
├── types/index.ts            Chambre, Demande, Contrat (+Ticket), constants
├── api/index.ts              real calls + mock implementations + domain events
├── mocks/index.ts            in-memory DB (rooms, demandes, contrats, tickets)
├── hooks/index.ts            useViewer, useChambres(Disponibles), useDemandes,
│                             useContrats(Visibles), useMesContrats, useTickets
├── schemas/index.ts          zod validators
├── i18n/index.ts             FR messages
├── utils/time.ts             toISODate, todayISO, addDays, formatDay, defaultDeparture
├── utils/students.ts         matricule → display name (demo directory)
```

```
├── components/
│   ├── ChambreCard.tsx            room card (+admin actions)
│   ├── ChambrePicker.tsx          available-room select grouped by bâtiment
│   ├── ChambreFormDialog.tsx      admin create-room dialog
│   ├── BatimentOccupancy.tsx      occupancy bars per building
│   ├── DemandeStatusBadge.tsx     EN_ATTENTE/APPROUVEE/REFUSEE/ANNULEE
│   ├── ContratStatusBadge.tsx     ACTIF/TERMINE/RESILIE
│   └── TicketStatusBadge.tsx      open/in_progress/resolved
└── pages/
    ├── HousingModule.tsx          tabs + nested routes
    ├── ResidencesPage.tsx         dashboard
    ├── ChambresPage.tsx           list/filters/admin CRUD
    ├── DemandesPage.tsx           student + admin views
    ├── NewDemandePage.tsx         demande form
    ├── ContratsPage.tsx           student + admin views
    └── MaintenancePage.tsx        tickets (adapted: residence → bâtiment)
```

Removed from the original scaffold (replaced by the above): `ResidenceCard.tsx`, `RoomPicker.tsx`, `HousingRequestStepper.tsx`, `OccupancyChart.tsx`, `MyRequestPage.tsx`, `RequestsPage.tsx`; old types (`Residence`, `HousingRoom`, `HousingRequest`) and old API routes (`/residences`, `/housing-requests`) are gone.

6. Current state and verification

Done: backend 3 services (validation, Swagger, events, pricing, academic-year rule, atomic occupation, schema bootstrap, docker-compose, env files); frontend module fully rewritten (role-aware pages, mock + real modes, FR i18n, README); docs (French technical PDF + this AI-context PDF/Markdown).

> **Verified:** backend `npm run build` OK · `npm test` = **36/36 tests** (Chambres 11, Demandes 12, Contrats 13) · `npm run lint` OK · frontend `npm run typecheck` OK · `npm run build` OK (Vite chunk `HousingModule` generated).

> **Not verified yet:** no real PostgreSQL / HTTP end-to-end run (Docker absent, no local Postgres; unit tests mock repositories) · JWT validation is a stub (any well-formed Bearer passes; roles enforced only in frontend) · contract creation after approval is eventual (listener logs errors instead of failing the approval).

7. Known limitations and risks

1. `DB_SYNCHRONIZE=true` is dev-only; no migrations yet.
2. Event listener failures are logged but not retried (no outbox); an approval could theoretically end with no contract.
3. Enum values are French uppercase in JSON — any new consumer must match exactly.
4. Backend dates are `timestamptz` ISO strings; the frontend sends `YYYY-MM-DD` (accepted by `@IsDateString`); the UI formats via local `fr-FR` formatters.
5. Maintenance tickets are out of scope for the three services (mock-only; explicit 501 in real mode).
6. No frontend test harness; no backend e2e tests.
7. `backend/` and all frontend changes are **uncommitted** (repo baseline = one teammates' commit).
8. CORS is wide-open (`origin: true`) for dev.
9. `NumericTransformer.from` assumes non-null; fine while `montantMensuel` is not nullable.

8. Roadmap — how to continue

#	Task	Concrete targets
T1	**Run the real stack once**	`cd backend && docker compose up -d` (or local PG) → `npm run start:dev` → exercise flows in Swagger: create rooms → POST demande → PATCH APPROUVEE → verify contract + room occupied → PATCH resilier. Then front `.env` : `VITE_USE_MOCKS=false` , `VITE_API_BASE_URL=http://localhost:3001/api`
T2	**Wire real authentication**	Implement verification in `backend/src/common/guards/jwt-auth.guard.ts` (shared secret/JWKS from `IAuthentification`), register as `APP_GUARD` in `app.module.ts` , mark Swagger/health as `@Public()` , map roles, derive `etudiantId` from token claims in `demandes.controller.ts` /service instead of the request body, add admin-only guards on chambre CRUD and status PATCH
T3	**Make approval atomic**	Use `DataSource.transaction` across `occuper` + contract insert + demande save (single shared DB) **or** add an outbox/retry for `demande.approuvee` ; compensate room occupation if saving the demande fails
T4	**Quality**	Backend e2e tests (`supertest` + test DB or Testcontainers); replace `synchronize` with TypeORM migrations; frontend test harness (Vitest + Testing Library)
T5	**Optional service split**	Extract the 3 modules into separate Nest apps + gateway; replace DI calls with HTTP clients and the event bus with a broker; add the maintenance service
T6	**Frontend polish**	Contract PDF download, richer admin dashboard, room deep-link detail, better empty/error states for real mode

9. Glossary and gotchas

French term	English	Notes
Chambre	Room	`disponible: true` = free; capacity depends on type
Bâtiment / Étage	Building / Floor	free-text building, `etage` ≥ 0
Demande d'hébergement	Housing request	`EN_ATTENTE` , `APPROUVEE` , `REFUSEE` , `ANNULEE`
Résident / Étudiant	Resident / Student	`etudiantId` (matricule, e.g. `ETU-2024-0187`)
Contrat	Contract	`ACTIF` , `TERMINE` , `RESILIE`
Montant mensuel	Monthly rent	TND; SIMPLE 150 / DOUBLE 100 / STUDIO 200
Année académique	Academic year	ends July 31; starts Aug 1
Signalement / Panne	Ticket / Breakdown	maintenance, mock-only

Gotchas for an AI working on this code:

- Status/type values are **French uppercase strings** in JSON — do not translate them in payloads.
- `chambreId` is a plain UUID column (no TypeORM relation) — existence is validated via `ChambresService.findOne` .
- One room hosts one resident: never set `disponible=false` directly; use `occuper()` semantics.
- Mock and backend must stay in sync: when changing a rule, update `backend/src/**/service.ts` , `src/features/housing/api/index.ts` mocks, README and specs.

- `Viewer` (`{ id, name, role }`) is a frontend-only concept for mocks/UI; the backend currently trusts `etudiantId` from the body.
- Do not touch other features' folders; the housing module is self-contained (the registry already routes it).

10. Command cheatsheet

```
# Backend
cd backend
npm install
docker compose up -d          # or provide a local PostgreSQL matching .env
npm run start:dev             # http://localhost:3001/api  ·  Swagger /api/docs
npm test                      # 36 unit tests
npm run lint && npm run build

# Frontend
npm install
npm run dev                   # http://localhost:5173 (mock mode by default)
npm run typecheck && npm run build

# Frontend against the backend (.env at repo root)
VITE_USE_MOCKS=false
VITE_API_BASE_URL=http://localhost:3001/api
```

End of context document — generated 2026-11 for the SmartCampus Résidences module.